eTapin Kigali

Smart Public Transport Demand Coordination System Documentation

1. Project Overview

eTapin Kigali is a smart public transport MVP designed to improve passenger-to-driver coordination in Kigali City.

The system helps passengers request transport service from a selected origin to a selected destination. Drivers can view passenger demand on their dashboard, identify high-demand stops, and manage boarding status.

The main purpose of eTapin Kigali is to prove a simple but powerful idea:

Passengers should be able to digitally signal where they are waiting and where they want to go, while drivers receive organized demand information that helps them respond better.

The current MVP focuses on:

- Passenger route search
- Passenger waiting request submission
- Driver demand dashboard
- Route and stop-based request aggregation
- Bus location simulation
- Passenger request status management

This documentation is based on the current eTapin Kigali implementation and project structure.

2. Project Objectives

2.1 Main Objective

To build a smart public transport coordination platform that connects passenger demand with available buses and drivers in a simple, organized, and real-time-friendly way.

2.2 Specific Objectives

The system aims to:

1. Help passengers find available routes between two stations.
2. Allow passengers to submit waiting requests.
3. Help drivers see where passengers are waiting.
4. Show demand pressure by route and stop.
5. Allow drivers to mark requests as boarded or cleared.
6. Simulate bus movement from one stop to another.
7. Provide a foundation for future real-time public transport management.

3. Problem Statement

Public transport passengers often face challenges such as:

- Not knowing whether a bus is available for their direction
- Waiting at stops without communicating demand
- Drivers not knowing where demand is highest
- Poor visibility of passenger concentration at stops
- Lack of a simple digital coordination system between passengers and transport operators

eTapin Kigali addresses these problems by creating a digital demand-signaling loop between passengers and drivers.

4. Proposed Solution

eTapin Kigali provides a web-based platform with two main user sides:

4.1 Passenger Side

Passengers can:

- Select their origin station
- Select their destination station
- Search for available routes
- View matching routes
- Submit a waiting request
- Cancel their request if needed

4.2 Driver Side

Drivers can:

- View total waiting passengers
- View waiting passengers by route
- View waiting passengers by stop
- Identify high-demand stops
- Mark passengers as boarded
- Clear or cancel waiting requests
- Simulate bus movement to the next stop

5. System Scope

5.1 Included in the Current MVP

The current version includes:

- Landing page

- Passenger portal
- Driver dashboard
- Station listing
- Route matching
- Passenger request creation
- Passenger request cancellation
- Driver request management
- Bus movement simulation
- Route-based demand aggregation
- Stop-based demand display

5.2 Not Included in the Current MVP

The current MVP does not include:

- User authentication
- Admin dashboard
- Live GPS tracking
- Real map integration
- Mobile application
- ETA prediction
- Notifications
- Route optimization engine
- Multi-city deployment
- Advanced analytics dashboard

6. Technology Stack

6.1 Frontend Technologies

The frontend is built using:

- **Next.js 14**
- **React 18**
- **TypeScript**

- **Framer Motion**
- **Lucide React**

6.2 Backend Technologies

The backend uses:

- **Next.js Route Handlers**
- **Prisma ORM**
- **SQLite database**

6.3 Development Tools

The development environment includes:

- **Node.js**
- **npm**
- **ESLint**
- **Prisma**
- **TSX seed scripts**

7. System Architecture

The system follows a modern full-stack web application structure.

7.1 Frontend Layer

The frontend provides user interfaces for:

- Landing page
- Passenger route search
- Passenger request submission
- Driver dashboard
- Bus movement simulation controls

7.2 Backend Layer

The backend handles:

- Station data retrieval
- Route search
- Passenger request validation
- Request status updates
- Driver dashboard data aggregation
- Bus position update simulation

7.3 Database Layer

The database stores:

- Stations
- Routes
- Route-stop sequences
- Passenger requests
- Buses

8. Repository Structure

```
src/
  app/
    page.tsx
    passenger/page.tsx
    driver/page.tsx

    api/
      stations/route.ts
      routes/route.ts
      passenger-requests/route.ts
      passenger-requests/[requestId]/route.ts
      driver-dashboard/route.ts
      buses/[busId]/advance/route.ts
```

```
  components/
    PassengerForm.tsx
    DriverDashboard.tsx

  lib/
    driverDashboard.ts

prisma/
  schema.prisma
  seed.ts
```

9. User Roles

9.1 Passenger

A passenger is a user who wants to find a route and submit a waiting request.

Passenger actions include:

- Open the passenger portal
- Select origin
- Select destination
- Search route
- View matching route
- Submit waiting request
- Cancel request

9.2 Driver

A driver is a user who views passenger demand and manages passenger boarding status.

Driver actions include:

- Open driver dashboard
- View waiting requests

- View route pressure
- View stop hotspots
- Mark request as boarded
- Clear request
- Move bus to next stop

10. Passenger Flow

Step 1: Open the Platform

The passenger opens the landing page.

Step 2: Search or Select Station

The passenger may search for a station or proceed directly to the passenger portal.

Step 3: Choose Origin and Destination

The passenger selects:

- Origin station
- Destination station

The system prevents selecting the same station as both origin and destination.

Step 4: Search Routes

The system searches for active routes that can serve the selected direction.

A valid route must:

- Contain the origin station
- Contain the destination station
- Have the origin before the destination in route order
- Have at least one active bus

Step 5: View Route Details

The passenger can view route information such as:

- Route name
- Stops
- Bus current location
- Travel direction context
- Route progress visualization

Step 6: Submit Waiting Request

The passenger submits a request showing that they are waiting at the selected origin station.

Step 7: Cancel Request

The passenger can cancel the request if they no longer need the service.

11. Driver Flow

Step 1: Open Driver Dashboard

The driver opens the driver portal.

Step 2: View Route Pressure

The dashboard displays:

- Total waiting requests
- Active buses
- Route count
- Highest-pressure route

Step 3: View Route Cards

Each route card shows:

- Active buses
- Current bus stations
- Waiting count per stop
- Hotspot stops
- Next bus stop

Step 4: Manage Passenger Requests

The driver can:

- Mark a passenger as boarded
- Clear a request
- View origin and destination of waiting passengers

Step 5: Advance Bus Position

The driver can move the bus to the next stop in the route sequence for simulation purposes.

12. Core Features

12.1 Landing Page Features

The landing page includes:

- Multilingual hero content
- Station-aware search
- Direct access to passenger portal
- Direct access to driver dashboard
- Route prefill support through query parameters

Supported languages include:

- English
- Kinyarwanda
- French
- Swahili

12.2 Passenger Portal Features

The passenger portal includes:

- Origin selector
- Destination selector
- Grouped station display
- Duplicate station prevention
- Route search
- Loading state
- Error handling
- Route cards
- Route detail expansion
- Waiting request submission
- Request cancellation
- Per-route request status feedback

Passenger request statuses include:

- Submitting
- Done
- Cancelling
- Cancelled

12.3 Driver Dashboard Features

The driver dashboard includes:

- Auto-refresh every 20 seconds

- Manual refresh button
- KPI tiles
- Route demand summaries
- Stop-level waiting counts
- Bus location display
- Next-stop simulation control
- Hotspot ranking
- Recent waiting request list
- Request action buttons

12.4 Backend Features

The backend provides:

- Station catalog API
- Route matching API
- Passenger request creation
- Passenger request status update
- Driver dashboard aggregation
- Bus movement simulation
- Request validation
- Route viability checking

13. Database Design

13.1 Station Model

The **Station** model represents both major hubs and normal bus stops.

Fields

```
id
name
latitude
```

```
longitude
placeId
isHub
```

Relationships

A station can:

- Belong to many route stops
- Be used as a passenger origin
- Be used as a passenger destination
- Be the current position of a bus

13.2 Route Model

The **Route** model represents a public transport line.

Fields

```
id
name
description
```

Relationships

A route can have:

- Many ordered stops
- Many buses

13.3 RouteStop Model

The **RouteStop** model connects routes with stations.

Fields

```
id
routeId
stationId
order
```

The `order` field is important because it defines the direction and sequence of the route.

Example:

```
Downtown -> KBC -> Gishushu -> Kisimenti -> Remera
```

13.4 PassengerRequest Model

The **PassengerRequest** model stores passenger demand.

Fields

```
id
stationId
destinationId
status
timestamp
```

Status Values

```
WAITING
BOARDED
CANCELLED
```

13.5 Bus Model

The **Bus** model represents a bus assigned to a route.

Fields

```
id
routeId
currentStationId
capacity
active
```

14. API Documentation

14.1 Get Stations

```
GET /api/stations
```

Purpose

Returns all stations grouped into hubs and stops.

Success Response

```
{
  "hubs": [
    {
      "id": "station-id",
      "name": "Nyabugogo Bus Park",
      "isHub": true
    }
  ],
  "stops": [
    {
      "id": "station-id",
      "name": "Gishushu",
      "isHub": false
    }
  ]
}
```

Possible Error

```
500 Server/database failure
```

14.2 Search Routes

```
GET /api/routes?originId=...&destinationId=...
```

Purpose

Returns active routes that can serve a passenger from origin to destination.

Validation Rules

- Origin is required
- Destination is required
- Origin and destination must be different
- Origin must come before destination
- Route must have at least one active bus

Possible Errors

```
400 Missing query parameters
500 Server/database failure
```

14.3 Create Passenger Request

```
POST /api/passenger-requests
```

Purpose

Creates a waiting request for a passenger.

Request Body

```
{
  "originId": "station-id",
  "destinationId": "station-id"
}
```

Success Response

```
{
  "requestId": "request-id",
  "waitingCount": 3,
  "originName": "Nyabugogo Bus Park",
  "destinationName": "Kimironko Bus Park",
  "message": "Passenger request created successfully."
}
```

Possible Errors

```
400 Missing origin or destination
400 Origin equals destination
404 Station not found
422 No active route can serve direction
500 Server/database failure
```

14.4 Update Passenger Request Status

```
PATCH /api/passenger-requests/[requestId]
```

Purpose

Updates the status of a passenger request.

Request Body

```
{
  "status": "BOARDED"
}
```

Allowed Statuses

```
WAITING
BOARDED
CANCELLED
```

Success Response

```
{
  "id": "request-id",
  "status": "BOARDED"
}
```

Possible Errors

```
400 Missing request id
400 Invalid status
404 Request not found
500 Server/database failure
```

14.5 Get Driver Dashboard

```
GET /api/driver-dashboard
```

Purpose

Returns the full driver dashboard data.

Returned Information

The response includes:

- Generation timestamp
- Total waiting requests
- Route summaries
- Active buses
- Current bus positions
- Stop demand counts

- Hotspot stops
- Recent waiting requests
- Recommended routes for requests

Possible Error

```
500 Aggregation failure
```

14.6 Advance Bus Position

```
PATCH /api/buses/[busId]/advance
```

Purpose

Moves a bus to the next stop in its route sequence.

Behavior

The system:

1. Finds the bus.
2. Finds the current stop.
3. Gets the next stop.
4. Updates the bus current station.
5. If the bus reaches the final stop, it loops back to the first stop.

Success Response

```
{
  "id": "bus-id",
  "currentStationId": "station-id",
  "currentStationName": "Gishushu"
}
```

Possible Errors

```
400 Missing bus id
400 Route has no configured stops
```

```
404 Bus not found
500 Server/database failure
```

15. Driver Dashboard Aggregation Logic

The driver dashboard aggregation is one of the most important parts of the system.

The logic performs the following operations:

1. Fetches all routes with active buses.
2. Fetches ordered stops for each route.
3. Fetches all waiting passenger requests.
4. Checks whether each request is compatible with each route.
5. Counts waiting passengers per route.
6. Counts waiting passengers per stop.
7. Finds bus current position.
8. Calculates the next stop for each bus.
9. Creates hotspot ranking.
10. Builds recent request list.
11. Suggests matching routes for recent requests.

This creates a clear operational view for the driver.

16. Seed Data

The demo database includes sample transport data for Kigali.

16.1 Major Hubs

```
Nyabugogo Bus Park
Remera Bus Park
Kimironko Bus Park
Downtown Station
```

16.2 Intermediate Stops

```
KBC
Gishushu
Kisimenti
Rwandex
Sonatubes
Giporoso
```

16.3 Demo Routes

Route 104

```
Downtown -> KBC -> Gishushu -> Kisimenti -> Remera
```

Route 201

```
Nyabugogo -> KBC -> Gishushu -> Kimironko
```

Route 305

```
Downtown -> Rwandex -> Sonatubes -> Giporoso
```

16.4 Demo Buses

The seed data includes three active buses with initial route positions.

17. Setup and Installation Guide

17.1 Requirements

Before running the project, install:

```
Node.js 18 or above
npm
```

17.2 Install Dependencies

Run:

```
npm install
```

17.3 Seed the Database

Run:

```
npx prisma db seed
```

17.4 Start Development Server

Run:

```
npm run dev
```

Then open the local development URL in your browser.

17.5 Build Production Version

Run:

```
npm run build
```

18. Current MVP Limitations

The current MVP is strong for demonstration, but it still has limitations.

Current limitations include:

1. No authentication system
2. No role-based access control
3. No admin dashboard
4. No live GPS tracking
5. No real map rendering
6. No real-time socket communication
7. No ETA prediction
8. No automatic bus telemetry
9. No route optimization engine
10. No mobile application
11. No notification system
12. No advanced analytics

19. Recommended Future Improvements

19.1 Security and Access Control

Recommended features:

- Passenger login
- Driver login
- Admin login
- Role-based access control
- Secure API access
- Protected dashboard routes
- Activity logs

19.2 Real-Time System Improvements

Recommended features:

- WebSocket or Server-Sent Events
- Real-time passenger request updates
- Real-time driver dashboard updates

- Real-time bus movement updates
- Live operational alerts

19.3 Map and Location Improvements

Recommended features:

- Live map display
- Route path visualization
- Bus position markers
- Passenger stop markers
- Stop clustering
- Driver location tracking
- Geolocation support

19.4 Passenger Experience Improvements

Recommended features:

- Saved favorite routes
- Saved frequent stations
- One-tap re-request
- Request status tracking
- Bus approaching notification
- Multilingual interface improvement
- Accessibility mode
- Large text mode
- High-contrast mode
- Voice-assisted navigation

19.5 Driver Experience Improvements

Recommended features:

- Driver account
- Assigned bus display
- Assigned route display
- Driver shift management
- Driver handover workflow
- Capacity status update
- Delay reporting
- Incident reporting
- Driver-to-control communication

19.6 Admin and Operations Improvements

Recommended features:

- Admin dashboard
- Route management
- Station management
- Bus management
- Driver management
- Demand heatmap
- System-wide KPIs
- Operational reports
- Route activation and deactivation
- Peak-hour planning support

19.7 Data and Intelligence Improvements

Recommended features:

- Historical demand analytics
- Demand prediction
- High-demand zone detection
- Demand trend charts
- Waiting time analysis
- Stop performance scoring

- Route performance analysis
- Demand spike detection

19.8 Engineering Improvements

Recommended features:

- Input validation schemas
- API versioning
- OpenAPI documentation
- Unit tests
- Integration tests
- End-to-end tests
- CI/CD pipeline
- Logging system
- Error monitoring
- Rate limiting
- Database migration strategy

20. Suggested Implementation Milestones

Milestone 1: Security Foundation

Focus:

- Authentication
- Role-based access
- Protected pages
- Secure API endpoints
- Input validation

Expected outcome:

Passengers, drivers, and admins can safely access only the parts of the system meant for them.

Milestone 2: Real-Time Dashboard

Focus:

- Replace polling with real-time updates
- Add live request updates
- Improve driver dashboard refresh behavior
- Add real-time route pressure changes

Expected outcome:

Drivers can see passenger demand changes instantly without manual refresh.

Milestone 3: Map-Based Visibility

Focus:

- Add map rendering
- Display stations on map
- Display route lines
- Display buses on map
- Show current bus location visually

Expected outcome:

Passengers and drivers can understand transport movement visually.

Milestone 4: Operational Intelligence

Focus:

- Demand analytics
- Hotspot detection

- Demand prediction
- Stop performance analysis
- Route pressure trends

Expected outcome:

The system helps operators make smarter public transport decisions.

Milestone 5: Admin Control Center

Focus:

- Admin dashboard
- Station management
- Route management
- Bus management
- Driver management
- System reports

Expected outcome:

Transport managers can control and monitor the full system from one dashboard.

21. Use Case Summary

21.1 Passenger Use Cases

Use Case	Description
Select origin	Passenger chooses where they are starting from
Select destination	Passenger chooses where they want to go
Search route	System finds matching active routes
View route	Passenger views available route details

Submit request	Passenger sends waiting request
Cancel request	Passenger cancels waiting request

21.2 Driver Use Cases

Use Case	Description
View dashboard	Driver opens demand dashboard
View waiting passengers	Driver sees active passenger requests
View hotspots	Driver identifies high-demand stops
Board passenger	Driver marks request as boarded
Clear request	Driver cancels or clears request
Advance bus	Driver moves bus to next stop in simulation

21.3 Admin Future Use Cases

Use Case	Description
Manage stations	Admin creates, updates, or removes stations
Manage routes	Admin manages transport routes
Manage buses	Admin manages buses assigned to routes
Manage drivers	Admin manages driver accounts
View analytics	Admin views demand and performance reports
Monitor system	Admin monitors system-wide activity

22. System Validation Rules

22.1 Passenger Request Validation

The system checks that:

- Origin is provided
- Destination is provided
- Origin and destination are not the same
- Origin station exists
- Destination station exists
- At least one active route can serve that direction

22.2 Route Matching Validation

A route is valid only when:

- The route contains the origin
- The route contains the destination
- The origin appears before the destination
- The route has at least one active bus

22.3 Request Status Validation

Only these request statuses are accepted:

```
WAITING
BOARDED
CANCELLED
```

Any other status must be rejected by the API.

23. Glossary

Term	Meaning
Hub	A major public transport station with high activity
Stop	A smaller intermediate boarding or alighting point

Route	A defined transport path followed by buses
RouteStop	A station assigned to a specific position in a route
Passenger Request	A signal showing that a passenger is waiting
WAITING	The passenger is still waiting
BOARDED	The passenger has entered the bus
CANCELLED	The request has been cancelled or cleared
Hotspot	A stop with high passenger demand
Active Bus	A bus currently available on a route

24. Professional Project Summary

eTapin Kigali is a practical and innovative transport coordination MVP that demonstrates how digital passenger demand can improve public transport visibility.

The system already provides a strong foundation by connecting passenger requests with driver dashboards. It allows passengers to signal demand, drivers to see waiting passengers, and the system to organize transport information by route and stop.

The current MVP is suitable for demonstration, testing, and future expansion. With improvements such as authentication, real-time updates, live maps, admin control, and analytics, eTapin Kigali can grow into a powerful smart public transport coordination platform for Kigali and beyond.

25. Final Conclusion

eTapin Kigali is not just a route finder. It is a demand coordination system.

Its strongest value is that it helps answer three important public transport questions:

1. **Where are passengers waiting?**
2. **Where do they want to go?**
3. **Which routes and buses can respond to that demand?**

By solving these questions, the system creates a strong foundation for smarter, more organized, and more responsive public transport operations in Kigali.